

PORTADA

Plataforma de Simulación Universal para Robótica Industrial

RESUMEN

PARRAFO DE CIERRE:

De esta manera obtenemos una plataforma con una curva de aprendizaje baja, sin requerir de muchos recursos tanto económicos como de hardware, muy flexible, modular, escalable y customizable.

0. Índice

0. Índice.....	3
1. Introducción.....	5
• 1.1. Contexto y Planteamiento del problema.....	5
• 1.2. Objetivos del Proyecto.....	5
• 1.3. Requisitos y Especificaciones del Sistema.....	7
• 1.4 Definiciones.....	7
2. Estado del Arte y Selección Tecnológica.....	8
• 2.1. El Contexto de la Robótica Industrial y el Alto Coste.....	8
• 2.2. Alternativas Low-Cost y Plataformas Educativas.....	8
• 2.3. Simuladores y plataformas actuales.....	8
• 2.4. Justificación del Stack.....	10
3. Arquitectura del Sistema.....	12
• 3.1. Visión general del sistema.....	12
• 3.2. Arquitectura Modular y Desacoplada.....	12
• 3.3. Ecosistema ROS2: Mapa de nodos y comunicaciones.....	13
4. Diseño e Implementación Software y Hardware.....	14
• 4.1. Cinemática del robot (Pybullet).....	14
• 4.2. Middleware y Gestión de Estados (ROS2 Jazzy).....	14
• 4.3. Implementación del Gemelo Digital.....	16
• 4.4. Interfaz de Hardware y Puente de Comunicación (Arduino).....	17
• 4.5 Caso de Uso Práctico: DLC "3 en raya" (Tic-Tac-Toe).....	19
5. Casos de Uso y Aplicación Educativa.....	21
• 5.1. Flujo de trabajo del usuario vs. desarrollador.....	21
• 5.2. Escalabilidad mediante paquetes.....	21
• 5.3. Relación con la teoría de Robótica Industrial y otras asignaturas.....	22
6. Pruebas y Resultados (Cubre el requisito: "resultados y pruebas").....	24
• 6.1. Pruebas de simulación y rendimiento.....	24
• 6.2. Pruebas de comunicación (Latencia).....	24
• 6.3. Validación de usuarios de prueba.....	24
7. Conclusiones y Trabajo Futuro (Cubre el requisito: "Conclusiones").....	24
• 7.1. Cumplimiento de objetivos y Conclusiones finales.....	24
• 7.2. Competencias adquiridas del Grado.....	24
• 7.3. Líneas de trabajo futuro.....	24
8. Bibliografía.....	24
9. Anexos.....	25

Tabla resumen de las imágenes

1. Introducción

● 1.1. Contexto y Planteamiento del problema

Durante el grado de Ingeniería en Robótica Software, los alumnos aprenden diferentes metodologías y herramientas para programar sistemas robóticos. Sin embargo, existe un campo fundamental que, por limitaciones de tiempo y recursos, se suele tratar de forma demasiado rápida y superficial: los brazos robóticos industriales.

En el plan de estudios, esta materia se concentra principalmente en el tercer año (en la asignatura de Robótica Industrial). En la práctica, esto se traduce en apenas un par de días de acceso al laboratorio para trabajar con un robot físico real. Además, debido al alto coste de estos equipos, es habitual que haya un mínimo de 8 personas asignadas a un único brazo robótico. Bajo estas condiciones, es casi imposible llegar a "trastear" y experimentar libremente con el equipo, lo que limita enormemente la capacidad de aprender de verdad mediante la práctica y el ensayo-error.

Este problema tiene una raíz lógica: las universidades públicas cuentan con un presupuesto limitado, lo que hace inviable comprar un brazo industrial para cada alumno o ampliar drásticamente las horas de laboratorio supervisado. A esto hay que sumarle que cada persona tiene un ritmo de aprendizaje distinto. Para asimilar un concepto de cinemática o control, un alumno puede necesitar 10 minutos y otro puede necesitar una hora de pruebas. Al depender exclusivamente del acceso a un hardware físico y compartido, la educación no se puede personalizar.

Por otro lado, las alternativas de software actuales tampoco terminan de solucionar el problema. Muchas plataformas son simuladores rígidos que obligan a trabajar o bien de forma puramente virtual, o bien de forma puramente física, sin un puente sencillo entre ambas. Además, si el alumno o el profesor quieren cambiar el modelo del robot para probar cosas nuevas (por ejemplo, usando herramientas estándar como MoveIt), se encuentran con procesos de configuración tediosos y complejos que quitan un tiempo valioso que debería dedicarse al aprendizaje.

● 1.2. Objetivos del Proyecto

Para dar solución a esta carencia en las aulas, se ha diseñado SURI. El objetivo general del proyecto es, como su propio nombre indica, crear una Plataforma de Simulación Universal para Robótica Industrial de bajo coste, orientada tanto al aprendizaje como al desarrollo.

Para alcanzar esta meta, el proyecto persigue los siguientes objetivos específicos:

Aplicación práctica de la teoría académica: Durante el primer bloque de la asignatura de Robótica Industrial se abordan conceptos teóricos densos y fundamentales (introducción a tareas robotizadas, morfología de manipuladores, cinemática directa, inversa y diferencial, y cálculo de singularidades). Al disponer de tan pocas horas prácticas con el robot real, estos temas suelen quedarse en el papel. El objetivo de SURI es servir de puente hacia la práctica: la plataforma emula funcionalidades operativas muy similares a las de un brazo industrial estándar (como los robots colaborativos UR), permitiendo a los alumnos aplicar la teoría programando acciones reales como el cambio del TCP (Tool Center Point) o la creación de paredes virtuales de seguridad

Crear una arquitectura altamente modular: La plataforma debe estar diseñada para adaptarse a cualquier situación o necesidad del usuario. Ya sea trabajando de forma puramente virtual o conectando un robot físico, cada componente del sistema debe ser independiente. Esta modularidad permite, por ejemplo, que si el día de mañana se quiere dejar de usar Arduino y cambiar a otro microcontrolador comercial, el núcleo del programa siga intacto. Bastaría con modificar o crear un nuevo nodo que sirva de puente de comunicación, sin perjudicar al resto del sistema.

Garantizar la facilidad de cambio de robot (Universalidad): A diferencia de otros softwares de control cinemático donde cambiar el robot implica generar paquetes de configuración enteros desde cero, SURI busca reducir este proceso a la mínima expresión. El objetivo es que el usuario solo tenga que proporcionar el archivo URDF del nuevo robot y ajustar un par de parámetros de configuración básicos para que el sistema lo asimile automáticamente. Así, el esfuerzo del estudiante recae en probar su código, no en configurar la herramienta.

Mantener un coste económico reducido: El gasto para replicar esta plataforma en la realidad debe ser mínimo. Al delegar la complejidad matemática al software (PC), el brazo físico puede construirse con materiales baratos y servomotores sencillos controlados por un simple Arduino, democratizando el acceso a la robótica física.

Asegurar la accesibilidad del hardware informático: El software de SURI no debe exigir ordenadores de última generación. Está pensado para que funcione fluidamente en los ordenadores estándar de los laboratorios de la universidad. Además, se busca que sea flexible a nivel de sistema operativo: aunque está diseñado para Linux, debe poder ejecutarse a través de herramientas como WSL (Windows Subsystem for Linux), evitando que los alumnos tengan que hacer particiones de disco en sus ordenadores personales si no saben hacerlo.

Alcanzar a una audiencia amplia: Aunque nace en el contexto de la Ingeniería en Robótica Software, la interfaz y el flujo de trabajo deben ser lo suficientemente amigables e intuitivos como para que personas de otras disciplinas (que no tengan conocimientos profundos de simuladores complejos) puedan acercarse a la robótica, controlar el brazo y entender sus fundamentos sin frustraciones.

● 1.3. Requisitos y Especificaciones del Sistema

Para garantizar el correcto funcionamiento de todos los módulos que componen SURI (simulador, motor de físicas y middleware de comunicación), el sistema requiere las siguientes especificaciones técnicas, tanto de software como de hardware.

Requisitos de Software:

- **Sistema Operativo:** Ubuntu 24.04 LTS (Nativo o emulado mediante WSL2 en Windows 10/11), debido a que es la versión empleada en los laboratorios de la universidad.
- **Middleware:** ROS2 Jazzy, como estándar de comunicación entre nodos.
- **Simulación y Físicas:** PyBullet (para el cálculo de cinemáticas y colisiones y para el entorno gráfico y el gemelo digital).
- **Control Físico:** Entorno de Arduino (IDE/CLI) para cargar el firmware a la placa.

Requisitos de Hardware (Ordenador del usuario): Dado que el ordenador debe ejecutar el entorno de simulación 3D y los cálculos cinemáticos en tiempo real de forma simultánea, se establecen los siguientes requisitos mínimos y recomendados:

- **Procesador (CPU):** Mínimo equivalente a un Intel Core i7 para manejar sin cuellos de botella los diferentes nodos de ROS2 y las físicas.
- **Memoria RAM:** Mínimo 16 GB (Recomendado 32 GB, especialmente importante si se ejecuta a través de WSL2, ya que Windows y Linux compartirán recursos).
- **Almacenamiento:** Aproximadamente 35 GB de espacio libre para instalar Ubuntu, ROS2, y dependencias asociadas.
- **Conectividad:** Al menos un puerto USB tipo A o C funcional para establecer la comunicación por puerto serie con la placa Arduino.

● 1.4 Definiciones

Brazo robotico:

Gemelo digital:

Simulador:

DLC:

Ros2:

...

2. Estado del Arte y Selección Tecnológica

Para diseñar la plataforma SURI, ha sido necesario analizar el abanico de herramientas de software y hardware que se utilizan actualmente en el mundo de la robótica. El objetivo de este análisis no es buscar la herramienta más potente del mercado, sino la que mejor se adapte a los objetivos del proyecto: facilidad de uso, bajo consumo de recursos y modularidad.

- 2.1. El Contexto de la Robótica Industrial y el Alto Coste

En la actualidad el sector industrial está dominado por brazos robóticos avanzados y cobots (robots colaborativos) como los fabricados por marcas como Universal Robots (UR) o KUKA. Estos equipos son verdaderas maravillas de la ingeniería que ofrecen precisión milimétrica y una integración perfecta en cadenas de montaje. Sin embargo su enfoque puramente industrial los convierte en un problema cuando intentamos llevarlos al aula.

El principal obstáculo es la barrera económica. Un brazo UR básico cuesta miles de euros. Las universidades con presupuestos ajustados solo pueden permitirse adquirir uno o dos de estos equipos para todo un laboratorio. Esto genera el problema mencionado en la introducción de tener a un grupo numeroso de alumnos mirando cómo un solo compañero programa el robot. Al no poder manipular el hardware libremente por miedo a romper un equipo tan caro la curva de aprendizaje se estanca. SURI nace precisamente para romper esta barrera económica trasladando la lógica y el software de estos grandes robots industriales a un entorno virtual y físico que cualquier estudiante pueda tener en su propia mesa.

- 2.2. Alternativas Low-Cost y Plataformas Educativas

- 2.3. Simuladores y plataformas actuales.

Software de cinemáticas y físicas:

El motor encargado de calcular las trayectorias, las cinemáticas inversas y la detección de colisiones es el núcleo matemático del sistema.

MoveIt 2: Es la herramienta por excelencia y el estándar absoluto dentro del ecosistema ROS2. Su coordinación con el middleware es perfecta. Sin embargo, tiene una curva de aprendizaje excesivamente alta para estudiantes que se inician. Además, no cumple con nuestro objetivo de "cambio fácil de robot": para integrar un brazo nuevo no basta con un archivo URDF, sino que exige pasar por un asistente complejo y generar un paquete de configuración completo, lo que desvía la atención del usuario.

Pinocchio + FCL: Son librerías extremadamente eficientes para el cálculo de dinámicas y colisiones rígidas. No obstante, su implementación a nivel de código es matemáticamente profunda y su curva de aprendizaje opaca demasiado el objetivo educativo de la plataforma.

PyBullet: Es un motor de físicas muy reconocido por su facilidad de uso y su integración directa con Python. A nivel de cinemáticas, permite cargar y cambiar de robot dinámicamente utilizando únicamente su archivo URDF, sin necesidad de configuraciones extra.

Software de simulación (Gemelo Digital):

El simulador es el encargado de renderizar el entorno 3D y permitir al usuario visualizar el comportamiento del robot antes de enviarlo al hardware real.

Isaac Sim: Desarrollado por Nvidia, ofrece un fotorrealismo y unas físicas inigualables. El problema es que es un software extremadamente pesado que requiere tarjetas gráficas muy potentes (RTX), lo que lo hace incompatible con los ordenadores estándar de un laboratorio universitario.

MuJoCo: Es un simulador físico excelente y muy rápido, pero históricamente ha sido difícil de implementar, presentando una curva de aprendizaje muy empinada para proyectos modulares.

CoppeliaSim: Es una opción intermedia, pero tiene el inconveniente de que no es 100% de código abierto en todas sus variantes, lo que choca con la filosofía accesible del proyecto.

Gazebo: Es el simulador tradicional en el entorno ROS. Aunque está muy estandarizado, consume bastantes recursos del sistema y sus visuales no siempre son las más amigables si no se invierte mucho tiempo en configuración.

Webots: Es un simulador muy equilibrado, con buena calidad visual y fácil de aprender. Fue considerado inicialmente, pero añadir un software de renderizado externo supone un consumo extra de CPU/GPU.

PyBullet (Módulo visual): Además de calcular físicas, PyBullet incluye un renderizador 3D integrado. Aunque visualmente es más básico que Isaac Sim o Webots, es increíblemente ligero y muy fácil de implementar.

Sistema de control (Middleware):

Es el encargado de comunicar los diferentes sistemas, guarda los datos necesarios como las herramientas creadas, es donde se añaden paquetes de funcionalidades y casos de uso (como jugar al 3 en raya).

ROS2 (Robot Operating System): Es el estándar indiscutible en el desarrollo de software robótico genérico a nivel mundial. Permite estructurar el código en nodos independientes, facilitando la arquitectura modular que persigue este proyecto.

Hardware / Microcontroladores:

Es el encargado del control del brazo robótico físico real. Se comunica con ros2 mediante el puerto USB, tanto indicando en qué posición está el robot cómo para recibir a qué posición tiene que ir el robot.

Raspberry Pi: Es un microordenador muy potente, capaz de ejecutar sistemas operativos completos. Aunque es una opción excelente, su coste es más elevado y ofrece una potencia de computación que, para este caso concreto, resulta innecesaria.

Arduino: Es el microcontrolador por excelencia en el ámbito educativo. Es extremadamente barato, accesible y su interfaz es universalmente conocida.

● 2.4. Justificación del Stack

Tras el análisis anterior, dando prioridad a crear una plataforma con una curva de aprendizaje muy baja, que permita al estudiante centrarse en probar su código, sin requerir hardware informático de altas prestaciones.

Aunque algunas herramientas se repiten, el sistema está programado en módulos completamente separados e independientes.

Motor de físicas: PyBullet. Este módulo se encarga de la parte matemática: calcular las cinemáticas necesarias, evitar singularidades y detectar posibles colisiones en el espacio. Se ha elegido PyBullet para este trabajo porque es el que presenta la curva de aprendizaje menor frente a opciones como Pinocchio. Además, cumple a la perfección con la meta principal del proyecto: permite cambiar de robot de forma casi automática leyendo solo su archivo URDF.

Esto permite al usuario centrarse directamente en programar el comportamiento del proyecto y no en configurar un motor matemático.

Simulación (Gemelo Digital): PyBullet. Aunque el motor de físicas también sea PyBullet, la parte gráfica funciona como un módulo completamente separado dentro de la arquitectura de SURI. Este módulo es el encargado de crear el gemelo digital en pantalla, renderizar el entorno 3D, visualizar los movimientos del robot y añadir cámaras u objetos virtuales. Para este módulo se podrían haber elegido simuladores como Webots o Gazebo, pero debido al objetivo de asegurar que SURI pueda ejecutarse fluidamente en los ordenadores de los laboratorios (y teniendo en cuenta los impedimentos físicos y limitaciones de hardware de mi propio ordenador durante el desarrollo), se decidió utilizar también el renderizador visual de PyBullet. Es una opción mucho más ligera que cumple la función visual sin sobrecargar el procesador ni la memoria RAM.

Control (Middleware): ROS2 Jazzy. Se elige porque es el estándar indiscutible de la industria robótica, y su funcionamiento por nodos es lo que permite que SURI añada paquetes de funcionalidades y casos de uso de manera limpia y modular.

Motor real (Hardware): Arduino. Se podría haber usado una Raspberry Pi o cualquier otro controlador más avanzado, pero se ha elegido Arduino porque toda la computación pesada ya la realiza el ordenador. No hace falta la potencia de una Raspberry para algo tan directo, haciendo de Arduino la opción más lógica, barata y fácil de conseguir para replicar el proyecto.

De esta manera obtenemos una plataforma con una curva de aprendizaje baja, sin requerir de muchos recursos tanto económicos como de hardware, muy flexible, modular, escalable y customizable

3. Arquitectura del Sistema

● 3.1. Visión general del sistema.

Tal y como se ha mencionado en apartados anteriores, el sistema está diseñado teniendo en mente a dos públicos objetivos: el usuario final y el desarrollador. *(En la imagen siguiente se puede observar el diagrama de Casos de Uso para cada perfil).*

Para el usuario básico: La plataforma ofrece una interfaz para utilizar funcionalidades que ya vienen integradas. El usuario puede mover el robot a una posición de forma manual o automática, crear y asignar un TCP (Tool Center Point) para cambiar de herramienta, establecer paredes virtuales de seguridad para que el robot no choque con el entorno, jugar al 3 en raya, o programar rutinas de movimiento completas.

Para el desarrollador: La visión del sistema es ser un entorno de trabajo abierto. El desarrollador puede añadir nuevas funcionalidades, nuevos DLCs¹, cambiar el modelo de robot al instante o incluso sustituir el simulador o el microcontrolador. Gracias a que la estructura de SURI es totalmente modular, se pueden cambiar varias piezas del sistema sin romper el funcionamiento de la plataforma en su conjunto.

¹ Se diferencia entre la creación de funcionalidades y el desarrollo de DLCs, siendo las funcionalidades ampliaciones de paquetes ya existentes (como la implementación de un movimiento MoveC) y los DLCs módulos independientes que hacen uso de dichas capacidades, como un juego de tres en raya.

● 3.2. Arquitectura Modular y Desacoplada

Para conseguir que la plataforma sea tan flexible y permita estos cambios, el diseño no se apoya en arquitecturas de software excesivamente complejas, sino en un sistema totalmente desacoplado basado en paso de mensajes, aprovechando el ecosistema de ROS2.

El concepto fundamental de SURI es la separación de responsabilidades: la lógica central realiza los cálculos pertinentes y sube el resultado final a un topic de ROS2. A este núcleo central le es indiferente quién consume esa información o para qué se use. Quien se suscribe a ese topic puede ser el Arduino para mover motores reales, el gemelo digital para mover el modelo 3D en pantalla, o un script de pruebas.

Gracias a este enfoque, los módulos no se necesitan mutuamente para sobrevivir. SURI no colapsa si desconectas el simulador o si el Arduino se apaga. De hecho, la arquitectura es tan mínima que, para que el sistema funcione a nivel lógico, los únicos nodos estrictamente obligatorios son el nodo de movimiento principal (move_arm) y el nodo matemático (kinematics_node). Todo lo demás actúa como un añadido opcional.

● 3.3. Ecosistema ROS2: Mapa de nodos y comunicaciones

A continuación se muestra un diagrama detallado de los paquetes de ROS2 y sus nodos correspondientes, junto con la leyenda explicativa de sus conexiones. Como se puede observar, el ecosistema se divide en tres paquetes principales: **robot_pkg**, **robot_kinematics** y **robot_digital_twin**.

1. robot_pkg (Paquete Principal y Comunicaciones): Es el corazón del ecosistema. Contiene el nodo fundamental `move_arm`, además de los nodos que hacen de puente hacia las distintas interfaces, como el visualizador estándar de ROS (RViz) y el hardware físico (Arduino). Este paquete aloja las acciones principales de movimiento del brazo: `moveL` (movimiento lineal) y `moveJ` (movimiento articular). También expone los servicios necesarios para crear o destruir herramientas dinámicamente (TCPs). El topic más importante de todo el sistema se encuentra aquí: `robot_cmd`. Este topic funciona como el bus de datos central por donde viaja el estado en ángulos de cada articulación del robot, siendo el que leen los demás módulos a conveniencia.

2. robot_kinematics (Motor Matemático y Físicas): Este paquete contiene el nodo obligatorio `kinematics_node`, encargado del trabajo matemático pesado. Funciona utilizando PyBullet en modo **DIRECT** (sin interfaz gráfica), así los cálculos consumen los mínimos recursos posibles de la CPU. Este nodo expone los servicios de cinemática inversa y directa (`solve_ik` y `solve_id`). Para calcular estas trayectorias correctamente, el nodo tiene en cuenta el TCP que se le pase en ese momento. Además, es el encargado de gestionar internamente las colisiones y de añadir o eliminar las paredes virtuales de seguridad para limitar el movimiento del robot.

3. robot_digital_twin (Gemelo Digital y Visualización): Es el paquete encargado de crear la representación visual en 3D (el gemelo digital). Para ello utiliza PyBullet en modo **GUI** (con interfaz gráfica). Al funcionar bajo la arquitectura desacoplada explicada antes, este paquete es totalmente opcional y no afecta al núcleo, se puede apagar si el ordenador del usuario tiene pocos recursos o si solo se quiere usar el robot físico. Este nodo gestiona todo lo puramente visual: ofrece servicios para crear o borrar objetos en pantalla, simular cámaras RGBD para la visión artificial y renderizar el entorno. Está sincronizado en tiempo real con `robot_kinematics`, de manera que las paredes virtuales invisibles calculadas por el motor matemático se puedan visualizar gráficamente aquí, haciendo mucho más fácil su comprensión.

4. Diseño e Implementación Software y Hardware

● 4.1. Cinemática del robot (Pybullet).

El cálculo de la cinemática directa e inversa del brazo robótico recae íntegramente sobre el motor matemático de PyBullet.

Carga dinámica del robot (URDF): El primer paso del sistema es leer el archivo URDF proporcionado por el usuario. En lugar de tener la morfología del robot programada de forma rígida en el código, PyBullet utiliza una función de parseo que lee las articulaciones, límites, masas y mallas 3D directamente de este archivo. Esto es lo que permite cumplir el objetivo principal del proyecto: cambiar de robot al instante sin tener que reprogramar el sistema.

Cinemática Directa (DK): Consiste en calcular la posición espacial final (X, Y, Z y orientación) a partir de los ángulos actuales de los motores. PyBullet nos da la posición del último eslabón del robot (la "muñeca"). Sin embargo, el usuario no trabaja con la muñeca, sino con la punta de la herramienta instalada (el TCP). Por tanto, el código obtiene la matriz de transformación de la muñeca y la multiplica por la matriz de transformación del TCP, obteniendo así las coordenadas reales y exactas de la punta de la herramienta.

Cinemática Inversa (IK): Es el proceso contrario: dada una posición objetivo en el espacio, ¿qué ángulos deben tener los motores? Al igual que en la cinemática directa, hay que tener en cuenta la herramienta. El algoritmo primero sitúa virtualmente la punta de la herramienta en la posición y orientación deseadas por el usuario. Luego, calcula dónde debería quedar situada la muñeca del robot. Una vez obtenido este punto clave, se llama al solver de cinemática inversa de PyBullet para que calcule los ángulos necesarios. En este paso se incluye un sistema de seguridad: si tras el cálculo el robot no puede llegar a esa posición con un margen de error menor a 2 cm (umbral ajustable), o si el movimiento implica una singularidad matemática grave (rotaciones excesivas) o detecta colisión, el sistema bloquea la acción y devuelve un error de "movimiento imposible".

Paredes virtuales y prevención de colisiones: Para crear restricciones en el espacio de trabajo, el sistema genera dinámicamente geometrías de colisión (cajas rectangulares largas y finas) en las coordenadas solicitadas. Durante el cálculo de trayectorias, el algoritmo comprueba si existe algún punto de contacto entre esta pared y cualquier eslabón del robot o la herramienta actual. Si detecta una intersección, la trayectoria se invalida instantáneamente.

● 4.2. Middleware y Gestión de Estados (ROS2 Jazzy).

Este subsistema gestiona el ciclo de vida del robot, la comunicación y la generación de trayectorias. Está estructurado en varios nodos dentro del paquete principal:

Nodo move_arm (Gestor de trayectorias): Es el cerebro de los movimientos y mantiene el registro de la herramienta activa en cada momento. Calcula dos tipos de trayectorias, enviando siempre el resultado final (los ángulos) al tópico robot_cmd:

- **Movimiento Articular (MoveJ):** Calcula la posición final deseada y genera una trayectoria interpolando suavemente los ángulos desde el origen hasta el destino para evitar movimientos bruscos en los motores.
- **Movimiento Lineal (MoveL):** Traza una línea recta cartesiana entre la posición actual y la final. Para evitar desviaciones, subdivide esa línea en múltiples puntos intermedios y calcula la cinemática inversa para cada uno de ellos, obligando a la herramienta a moverse en línea recta perfecta.
- **Justificación académica:** Cabe destacar que dentro de este nodo se ha dejado programada y comentada una función para calcular la cinemática directa mediante las matrices de Denavit-Hartenberg (DH) tradicionales, multiplicando la tabla paso a paso. Se deja a disposición del estudiante para que pueda poner en práctica la teoría de clase, aunque por defecto el sistema usa PyBullet porque el método DH obligaría a reescribir la tabla cada vez que se cambie de modelo de robot, rompiendo la filosofía de SURI.

Nodo execute_routine: Actúa como un intérprete. Crea clientes para todos los servicios disponibles, lee las rutinas programadas por el usuario línea a línea y las va solicitando. Si algún comando da fallo (por ejemplo, intentar ir a un punto inalcanzable), la rutina se detiene automáticamente por seguridad.

Nodo rviz_bridge: Sirve de traductor. Recoge los ángulos en crudo del tópico robot_cmd y los traduce al estándar joint_state de ROS2. Esto es vital para que herramientas estándar como RViz o el Gemelo Digital puedan visualizar el movimiento en tiempo real.

Nodo servo_bridge: Es el adaptador de hardware que se comunica con la placa Arduino mediante el puerto serie. Envía las posiciones objetivo a los motores y recibe la telemetría real. También procesa comandos especiales, como cambiar el robot de modo automático a manual.

Para estandarizar la comunicación entre todos estos nodos y permitir la escalabilidad mediante "DLCs", se ha creado un paquete independiente llamado **robot_interfaces**, que define los siguientes servicios:

- AddCamera.srv / SpawnObject.srv: Añadir o eliminar cámaras y objetos 3D en el gemelo digital.
- AddObstacle.srv / RemoveObstacle.srv: Gestión de paredes virtuales de seguridad.
- ManageTool.srv: Creación, selección y borrado de herramientas (TCPs).
- SolveDK.srv / SolveIK.srv: Peticiones matemáticas puras de cinemática directa e inversa.

- PutPiece.srv: Servicio específico creado como ejemplo de escalabilidad (DLC del 3 en raya).

● 4.3. Implementación del Gemelo Digital.

El gemelo digital es una réplica virtual que refleja el estado físico del sistema en tiempo real. Su utilidad en robótica es incalculable: permite hacer pruebas de validación, prevenir colisiones y desarrollar rutinas complejas sin riesgo de romper equipos costosos y sin necesidad de estar físicamente en el laboratorio.

En SURI, este módulo se ejecuta a través de PyBullet en su modo visual (GUI). Se suscribe al tópico `joint_state` para imitar exactamente el movimiento de las articulaciones y se sincroniza con el motor cinemático para renderizar visualmente las paredes virtuales y objetos del entorno.

Una de las implementaciones más complejas e importantes de este módulo es la simulación de cámaras RGB-D (Visión Artificial). En lugar de ser un simple visor gráfico, el gemelo digital simula cámaras robóticas reales siguiendo los estándares de ROS2:

1. **Cálculo Matemático (Modelo Pinhole):** Al solicitar la creación de una cámara, el código utiliza la posición objetivo y el campo de visión (FOV) para calcular las matrices de vista y proyección. A partir de ahí, extrae los parámetros intrínsecos de la lente virtual (distancia focal y centro óptico).
2. **Streaming de datos:** Se crean publicadores dinámicos a 10Hz que emiten tres tipos de datos estándar: la imagen en color (`/rgb/image_raw`), el mapa de profundidad calculado en metros reales (`/depth/image_raw`), y la información de calibración de la cámara (`/camera_info`).
3. **Árbol de Transformadas (TF):** Para que los algoritmos de Visión Artificial funcionen correctamente, el código no solo publica la imagen, sino que inyecta en el sistema ROS2 la posición espacial de la cámara. Emite dinámicamente dos transformadas: el Link Frame (que sitúa físicamente la cámara en el mundo) y el Optical Frame, aplicando las rotaciones matemáticas necesarias (convención OpenCV) para que el eje Z apunte hacia el frente de la lente y el eje Y hacia abajo.

De esta manera, cualquier algoritmo de IA o visión artificial externo creerá que está comunicándose con una cámara física real conectada al robot.

● 4.4. Interfaz de Hardware y Puente de Comunicación (Arduino).

Para el control físico del brazo robótico, se ha seleccionado una placa Arduino UNO básica. La elección de este microcontrolador se debe a que es extremadamente asequible a nivel económico, está muy popularizado y es muy común encontrarlo en laboratorios escolares e incluso en las casas de los estudiantes.

Para establecer la comunicación entre Arduino y el ecosistema ROS2 en el ordenador, se ha decidido utilizar una comunicación clásica por Puerto Serie (USB) en lugar de integrar protocolos más modernos como micro-ROS. La justificación principal es la **modularidad y universalidad**. No todos los microcontroladores del mercado tienen la memoria o la capacidad para ejecutar micro-ROS de forma nativa. Al utilizar comunicación Serial plana, SURI garantiza que cualquier microcontrolador (ya sea un PIC, un STM32, un ESP32 o un Arduino básico) pueda usarse en el proyecto sin mayor complejidad, ya que todos disponen de un puerto serie universal. El usuario solo tendría que cambiar la placa, sin modificar el código de ROS2.

Descripción del sistema físico: El hardware montado consta de los siguientes elementos:

- **Servomotores:** Actuadores encargados de mover las articulaciones del brazo físico.
- **Potenciómetros:** Permiten al usuario mover cada servomotor de forma independiente y manual.
- **Botonera de modos:** Botones físicos para cambiar el estado operativo del robot (Modo Automático, Modo Manual y Modo Creación de Pared Virtual).
- **Feedback visual (3 LEDs):** Indican en todo momento al usuario en qué modo se encuentra la placa, algo fundamental por seguridad cuando se interactúa físicamente con robots. (Por ejemplo, es obligatorio estar en modo manual para poder activar el modo de creación de pared virtual).

Flujo de datos Bidireccional: Dado que el sistema tiene dos formas de operar (controlado por el ordenador o controlado a mano por el usuario), el nodo puente de ROS2 (servo_bridge) y el Arduino mantienen una comunicación bidireccional constante.

Para que la comunicación sea rápida y fácil de procesar, se utilizan cadenas de texto simples. Si se mandan posiciones de motores, el formato siempre es una lista separada por comas: "(angulo_1),(angulo_2),(angulo_3)...", escalable dinámicamente dependiendo de cuántos grados de libertad tenga el brazo configurado. Si se mandan instrucciones de estado, se usan comandos de texto plano como "AUTO", "MANUAL" o "WALL".

- **ROS2 → Arduino (Modo Automático):** En este modo, el ordenador toma el control. El nodo puente de ROS2 se suscribe al tópico robot_cmd. Cada vez que el motor de físicas calcula una nueva trayectoria, este nodo coge los ángulos y se los envía por puerto serie al Arduino en el formato de lista mencionado. El Arduino simplemente

recibe la cadena de texto, la trocea, y aplica esos ángulos a los servomotores correspondientes.

- **Arduino → ROS2 (Modo Manual):** En este modo, el usuario mueve el brazo físicamente usando los potenciómetros. El Arduino lee estos sensores y envía constantemente (cada 20 ms) la posición actual de los motores hacia el ordenador. El nodo puente en ROS2 recibe esta información y actualiza el estado del sistema. Esto tiene dos grandes beneficios:
 1. Permite visualizar en tiempo real los movimientos del robot físico dentro del Gemelo Digital en pantalla.
 2. Evita movimientos bruscos: si el usuario vuelve a cambiar al "Modo Automático", el sistema de ROS2 ya sabe exactamente en qué posición física se quedó el robot, pudiendo calcular la cinemática desde ese punto sin que el brazo dé un "salto" violento.

Caso de uso especial: Paredes Virtuales Físicas El sistema incluye una funcionalidad muy útil que mezcla el mundo real y el simulado. Estando en modo manual, el usuario puede pulsar el botón de "Pared Virtual". A partir de ese momento, el usuario mueve físicamente la punta del robot a un punto del espacio y pulsa el botón para registrarlo y el Arduino envía el comando al ordenador. Al recibir estos tres puntos espaciales, el motor matemático de SURI en ROS2 es capaz de calcular el plano geométrico que pasa por ellos y levanta una pared virtual de seguridad en el simulador. Así, se delega la interfaz al Arduino, pero el cálculo matemático pesado se realiza en el ordenador.

Configuración asíncrona del Puerto Serie (No Bloqueante): A nivel de implementación de software en el nodo de ROS2 (servo_bridge), gestionar la lectura del puerto serie requiere especial precaución. En lugar de usar métodos de lectura estándar, el puerto serie (por ejemplo, /dev/ttyACM0) se ha configurado directamente a nivel de sistema operativo Linux modificando sus flags para que opere en modo No Bloqueante (Non-Blocking) a una velocidad estable (57600 baudios, tiene que coincidir con lo configurado en arduino).

Esta justificación técnica es vital: si la comunicación fuera bloqueante y el cable de Arduino se desconectara por accidente, o la placa tardará en responder, el nodo de ROS2 se quedaría esperando el dato de forma infinita, rompiendo toda la simulación. Al configurarlo como asíncrono y no bloqueante, el nodo lee el puerto: si hay datos, los procesa y los publica, si no hay datos, el ciclo continúa inmediatamente, garantizando que PyBullet y el Gemelo Digital sigan funcionando a sus hercios (Hz) correspondientes sin interrupciones ni cuellos de botella.

A continuación se mostrará una imagen del conexionado hardware de arduino.

● 4.5 Caso de Uso Práctico: DLC "3 en raya" (Tic-Tac-Toe)

Para demostrar la gran capacidad, flexibilidad y potencial de esta plataforma, se ha desarrollado un paquete "DLC" complejo que agrupa los conocimientos adquiridos en diversas asignaturas de la carrera (justificación más detallada en el apartado 5.3).

Este paquete consiste en un juego de "3 en raya" donde un usuario humano juega contra el brazo robótico. Se ha implementado reutilizando los topics, servicios y acciones ya creados en el programa principal, demostrando que SURI es cien por cien modular. En esta iteración, el DLC está pensado para ejecutarse exclusivamente en el entorno de simulación, ya que requiere el uso de una cámara RGB-D cenital, un hardware costoso de replicar físicamente para un usuario medio. Para que el juego sea visual, el paquete incluye sus propios archivos URDF con los modelos 3D del tablero y las fichas (X y O) para inyectarlos en la simulación.

El ciclo de ejecución de este paquete se divide en varias fases secuenciales integradas en un único nodo de ROS2:

1. Visión Artificial y Procesamiento de Imagen (OpenCV) El sistema se suscribe a los topics de imagen a color e imagen de profundidad de la cámara virtual de SURI, tratándolos como si provinieran de una cámara física real. El primer reto es aislar el área de juego. Usando la librería OpenCV, el nodo aplica filtros de escala de grises y detección de bordes (Canny) para encontrar los contornos del tablero. Una vez identificadas las 4 esquinas reales en la imagen, se aplica una transformación de perspectiva para aplanar la imagen y corregir la distorsión geométrica. Finalmente, la imagen corregida se recorta y se divide matemáticamente en una cuadrícula de 9 casillas individuales, que se guardan temporalmente en el sistema para su análisis.

2. Reconocimiento de Patrones mediante Red Neuronal (PyTorch) Una vez divididas las casillas, el sistema necesita saber qué hay en ellas para reconstruir el estado de la partida. Para ello, se pasa cada imagen a un modelo de Inteligencia Artificial.

- **Modelo CNN:** Se ha diseñado y entrenado desde cero una Red Neuronal Convolutiva (CNN). La red cuenta con varias capas de extracción de características (convolución y *Max Pooling*), seguidas de capas lineales de clasificación. Se ha incluido una capa de *Dropout* (apagado aleatorio de neuronas) durante el entrenamiento para evitar que el modelo memorice las imágenes y mejore su generalización frente a variaciones de luz o posición. El modelo devuelve si la casilla contiene una 'X', una 'O', o si está vacía. Esto permite que se pueda crear otro modelo de imágenes más complejas sin necesidad de montar de nuevo todo el código, solo reentrenando.
- **Automatización de Entorno (Usabilidad):** Levantar un entorno de IA suele ser complicado para usuarios nuevos debido a los conflictos de librerías. Para cumplir con el objetivo de "alcanzar a una audiencia amplia", se ha programado un script de Bash (`init_python.sh`). Este script automatiza la creación de un entorno virtual de Python,

instala una versión ligera de PyTorch (solo CPU, ahorrando más de 3GB de espacio) e instala las dependencias necesarias. Gracias a esto, cualquier persona puede ejecutar la IA de reconocimiento visual, sin requerir conocimientos previos de administración de sistemas.

3. Toma de Decisiones (Algoritmo IA - Minimax) Con el estado del tablero reconstruido virtualmente, es el turno del robot. Para garantizar un comportamiento inteligente, la lógica del juego utiliza el algoritmo *Minimax*. El código analiza el árbol completo de posibilidades futuras, evaluando todas las jugadas legales que quedan en el tablero. Asigna una puntuación positiva si el movimiento lleva a la victoria del robot, negativa si lleva a la victoria del humano, y neutra en caso de empate. Al elegir siempre el camino óptimo, este algoritmo garantiza matemáticamente que el brazo robótico jamás perderá una partida: sus únicos resultados posibles son ganar o forzar un empate.

4. Transformación Espacial y Ejecución Cinemática Una vez que el algoritmo elige la casilla óptima para colocar su ficha, el nodo debe traducir el píxel 2D de la imagen al espacio 3D real del robot. Haciendo uso del mapa de profundidad de la cámara y del árbol de transformadas de ROS2 (TF2), el sistema extrae las coordenadas X, Y, Z exactas de esa casilla referenciadas a la base del robot.

A continuación, el nodo solicita a SURI ejecutar los movimientos:

- Manda una acción *moveJ* (movimiento articular) para que el brazo se acerque de forma segura al punto superior de la casilla objetivo.
- Manda una acción *moveL* (movimiento lineal) para bajar en línea recta perfecta.
- Se llama al servicio de instanciación para "soltar" (*spawnear*) la ficha correspondiente en el tablero.
- Se retira el brazo de nuevo hacia arriba.

5. Gestión de Errores y Ciclo de Juego Durante los movimientos físicos, el sistema evalúa constantemente el éxito de la cinemática. Si el punto calculado para colocar la ficha resulta estar fuera del alcance máximo del brazo robótico (o si se detecta una singularidad/colisión inminente), el programa intercepta el error, detiene la rutina por seguridad y avisa por consola. Esto permite al usuario humano recolocar el tablero virtual más cerca y reiniciar la partida automáticamente. Si el movimiento se completa con éxito, el sistema comprueba si hay ganador; si no lo hay, cede el turno al usuario, manteniéndose en un bucle de espera activa evaluando la cámara hasta que detecta una nueva jugada humana.

5. Casos de Uso y Aplicación Educativa

● 5.1. Flujo de trabajo del usuario vs. desarrollador.

Este proyecto se ha diseñado pensando en dos perfiles de uso completamente distintos: el usuario básico y el desarrollador.

El **perfil de usuario** es el público objetivo más amplio. Está pensado para estudiantes de primeros cursos o personas sin conocimientos profundos de robótica o programación. Para este perfil, la plataforma funciona "out of the box" (lista para usar): solo tienen que lanzar el programa y pueden ejecutar tareas o funcionalidades que ya están preprogramadas, interactuando con el robot de forma visual e intuitiva.

Por otro lado, el **perfil de desarrollador** está enfocado a estudiantes avanzados de la carrera (como los propios alumnos de Ingeniería en Robótica Software). Para ellos, SURI no es solo un programa cerrado, sino un entorno de pruebas. Ofrece libertad total para meterse en el código, modificar comportamientos o añadir nuevos sensores.

Hacer esta diferenciación es clave para cumplir con uno de los objetivos principales del proyecto: alcanzar a una audiencia amplia. Obviamente, una persona con conocimientos avanzados también puede limitarse a usar la plataforma como usuario básico si solo quiere hacer una prueba rápida. (Para ver el paso a paso de cómo se arranca y utiliza el sistema en ambos perfiles, se puede consultar el *Anexo: Manual de Uso*).

● 5.2. Escalabilidad mediante paquetes.

Una de las grandes ventajas de SURI es su enorme modularidad. Esta arquitectura permite añadir nuevas funcionalidades al sistema como si fueran "DLCs" (contenido descargable adicional, como se les ha llamado a lo largo de este trabajo para entender el concepto).

Gracias a que todo el sistema de control funciona sobre ROS2, añadir un "DLC" no rompe el programa original. Si un alumno quiere que el robot haga algo nuevo, solo tiene que crear su propio paquete (nodo) y suscribirse a los *topics* que ya existen en SURI (por ejemplo, leer la posición actual del brazo o la imagen de una cámara) y publicar en los *topics* de control. SURI asimila este paquete nuevo sin tener que reprogramar el núcleo. (Para más detalles sobre un ejemplo real de cómo crear e integrar un DLC nuevo, ver el *Anexo: Manual de Uso: Añadir DLCs nuevos*).

- 5.3. Relación con la teoría de Robótica Industrial y otras asignaturas.

El desarrollo de SURI no es un elemento aislado, sino que sirve como aplicación práctica directa de los conocimientos adquiridos a lo largo de la carrera. A continuación se detalla cómo la plataforma integra el temario de varias asignaturas clave:

Robótica Industrial: Es la asignatura que motivó este proyecto. SURI aplica directamente la teoría a la práctica. En concreto, el motor físico calcula la cinemática directa, inversa y diferencial, y tiene en cuenta las singularidades (Tema 3 y Tema 2 del temario). Además, SURI incluye funcionalidades operativas idénticas a las de los robots colaborativos UR que se ven en el Tema 1, permitiendo a los alumnos hacer cosas como cambiar el TCP (Tool Center Point / Herramienta) dinámicamente o crear paredes virtuales para limitar el área de trabajo del robot.

Modelado y Simulación de Robots: SURI cubre prácticamente todos los bloques de esta asignatura. Se utiliza PyBullet como motor físico de cuerpos rígidos (Bloque II) y se integran representaciones 3D de robots mediante archivos URDF (Bloque III). Además, la plataforma es un ejemplo perfecto de cómo solucionar el gap "sim2real" (Bloque IV), al pasar el comportamiento simulado en el ordenador a un brazo físico real. Todo esto coordinado mediante ROS2, aplicando lo visto sobre integración de middlewares y simulación (Bloque V y Bloque I).

Arquitectura para Robots: La estructura interna de SURI refleja directamente el temario de esta asignatura. Se utiliza ROS2 como middleware de programación (Tema 2), organizando el software en base a nodos independientes para generar comportamientos robóticos (Tema 4) y permitiendo la creación de nuevas capacidades para los robots mediante paquetes (Tema 3).

Visión Artificial: Para demostrar la capacidad de SURI, se ha implementado un DLC en el que el robot juega al "3 en raya". Para ello, se usa una cámara virtual (o real) que aplica los conceptos de formación de imagen (Tema 2) y reconocimiento de patrones (Tema 10) para entender el estado del tablero.

Aprendizaje Automático e Inteligencia Artificial: Siguiendo con el DLC del "3 en raya", el sistema necesita saber cómo ganar y qué piezas hay en la mesa. Para ello se ha aplicado el algoritmo de IA *Min-Max* para la toma de decisiones. Además, se ha creado e integrado una Red Neuronal Convolucional (CNN) entrenada para clasificar si una imagen es una "X" o una "O", aplicando los conceptos de algoritmos de clasificación (Tema 5) y Redes Neuronales Artificiales (Tema 8) vistos en la asignatura de Aprendizaje Automático.

Mecatrónica Para el robot físico de SURI ha sido necesario realizar un diseño mecánico básico y la integración de microcontroladores y actuadores (servomotores) para construir el brazo robótico de bajo coste (Tema 4), requiriendo además nociones de programación cruzada (Tema 5) para comunicar el PC con el Arduino.

6. Pruebas y Resultados (Cubre el requisito: "resultados y pruebas")

- 6.1. Pruebas de simulación y rendimiento.
- 6.2. Pruebas de comunicación (Latencia).
- 6.3. Validación de usuarios de prueba.

7. Conclusiones y Trabajo Futuro (Cubre el requisito: "Conclusiones")

- 7.1. Cumplimiento de objetivos y Conclusiones finales.
- 7.2. Competencias adquiridas del Grado
- 7.3. Líneas de trabajo futuro.

cluir una reflexión, cuantitativa o cualitativa, sobre el posible impacto (positivo o negativo, directo o indirecto, actual o futuro) y las responsabilidades relacionadas con el TFG en lo relativo a: • Impacto social: grupos de interés afectados, accesibilidad, seguridad, privacidad, bienestar, igualdad de género, etc. • Impacto económico: viabilidad, mejora de la productividad, mantenimiento... • Impacto medioambiental: sostenibilidad, consumo energético y otros recursos naturales, contaminación, reciclaje, etc. • Responsabilidad ética y profesional: gestión y control de riesgos, respeto a las normas o regulaciones profesionales o legales, respeto a los derechos de propiedad intelectual, respeto a la legislación sobre protección de datos, respeto a los códigos deontológicos de la ingeniería, etc.

8. Bibliografía

9. Anexos